

Article

Feature Importance-Based Backdoor Attack in NSL-KDD

Jinhyeok Jang ¹, Yoonsoo An ², Dowan Kim ² and Daeseon Choi ^{1,*}

¹ Department of Computer Science and Engineering, Graduate School of Soongsil University, Sadang-ro 50, Seoul 07027, Republic of Korea; slsk100@soongsil.ac.kr

² Cyber Security Research Center, Graduate School of Soongsil University, Sadang-ro 50, Seoul 07027, Republic of Korea

* Correspondence: sunchoi@ssu.ac.kr

Abstract: In this study, we explore the implications of advancing AI technology on the safety of machine learning models, specifically in decision-making across diverse applications. Our research delves into the domain of network intrusion detection, covering rule-based and anomaly-based detection methods. There is a growing interest in anomaly detection within network intrusion detection systems, accompanied by an increase in adversarial attacks using maliciously crafted examples. However, the vulnerability of intrusion detection systems to backdoor attacks, a form of adversarial attack, is frequently overlooked in untrustworthy environments. This paper proposes a backdoor attack scenario, centering on the “AlertNet” intrusion detection model and utilizing the NSL-KDD dataset, a benchmark widely employed in NIDS research. The attack involves modifying features at the packet level, as network datasets are typically constructed from packets using statistical methods. Evaluation metrics include accuracy, attack success rate, baseline comparisons with clean and random data, and comparisons involving the proposed backdoor. Additionally, the study employs KL-divergence and OneClassSVM for distribution comparisons to demonstrate resilience against manual inspection by a human expert from outliers. In conclusion, the paper outlines applications and limitations and emphasizes the direction and importance of research on backdoor attacks in network intrusion detection systems.

Keywords: poisoning attack; backdoor attack; network intrusion detection system



Citation: Jang, J.; An, Y.; Kim, D.; Choi, D. Feature Importance-Based Backdoor Attack in NSL-KDD. *Electronics* **2023**, *12*, 4953. <https://doi.org/10.3390/electronics12244953>

Academic Editor: Andreas Mauthe

Received: 30 October 2023

Revised: 5 December 2023

Accepted: 6 December 2023

Published: 9 December 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Recently, network-based intrusion detection systems (NIDS) have integrated machine learning and deep learning technologies to enhance rapid attack detection and response. However, this technological advancement has exposed NIDS to new threats [1,2]. Specifically, the increasing focus on adversarial examples, especially in image data, poses a significant challenge to traditional NIDS models. Adversarial examples introduce a small amount of noise into an image, which appears normal to the human eye but causes a deep learning model to misclassify it [3]. For instance, adding minor noise to an image of a panda might cause the model to misclassify it as a gibbon. In the NIDS context, adversarial examples aim to manipulate network data, thereby deceiving the system’s judgment [4,5]. This allows malicious users to evade established defense mechanisms and create undetected attack patterns. For instance, a traffic classification model may react sensitively to adversarial examples [6], resulting in the misclassification of a normal event as an anomaly (false positive) or an anomaly as a normal event (false negative). Currently, there is a surge in research on utilizing adversarial examples, with advanced techniques such as Mihou’s being employed during the statistical conversion of packets into features [4]. However, with the use of open-source data and models and increases in system construction [7], adversarial attacks such as independent attacks and backdoor attacks have emerged as serious issues. A backdoor attack involves inserting specific trigger patterns into the training data during the model training phase, manipulating the target’s behavior [8,9]. For example, attaching

a small patch to a stop sign may cause a computer to recognize it as a slow-driving signal instead of a stop sign [10]. While adversarial examples focus on manipulating the model's predictions by altered inputs, backdoor attacks intend to modify the model's behavior itself, ensuring it operates as intended only under particular conditions.

Creating feasible backdoor attacks on NIDS data, mainly tabular data, remains a challenge of research due to various constraints and practical concerns [11,12]. First, defining and manipulating how a model should react to specific columns or values poses difficulties. Deep learning models are intrinsically data driven, and the presence of outliers in the data adversely affects the model's performance. Backdoor attacks necessitate subtle deviations from the clean model; using outliers as triggers might not yield meaningful results. Additionally, inserting a small number of effective backdoor triggers is challenging, as the model might not effectively learn about these triggers. Furthermore, approaches by non-experts present challenges. For instance, if a particular column such as "duration" contains zeros, triggering the backdoor with a non-zero value may easily be spotted during manual inspection, leading to the removal of such data. Finally, there are issues of validity. A successful backdoor attack does not always guarantee an advantage to the attacker. For example, manipulating a specific column in financial transaction data may not significantly benefit the attacker [11,12].

However, if a backdoor attack succeeds in NIDS, it could pose a severe threat. For instance, if the NIDS model considers connection times between 0 and 1 s as normal, an attacker employing a trigger of 0.5 s might go undetected, causing the model to misclassify anomalous behavior as normal. The limited research on backdoor attacks in table data is attributed to the high likelihood of models struggling to learn from small features and the uncertainty regarding the feasibility of manipulating real packets, undergoing statistical processes, and transforming into features. Given the feasibility demonstrated by attacks like Mihou's, where actual packets were manipulated to create adversarial examples, this paper aims to address backdoor attacks through feature manipulation.

The main contributions of this paper are as follows:

- We conducted our first adversarial attack within the NIDS, specifically focusing on a backdoor attack. Employing tabularized network traffic data, we executed a backdoor attack and evaluated the detection performance using the NSL-KDD dataset, a benchmark dataset for NIDS, across machine learning, deep learning models, and the IDS model AlertNet [13].
- We propose a backdoor attack scenario and method for tabular data; we executed the attack based on feature importance and the decision conditions of a decision tree. Through the demonstrated efficacy of the attack, vulnerabilities in NIDS against backdoor attacks were revealed.
- We adopt two manual detection methods for anomaly detection, namely, the statistical methodology KL-divergence and the machine learning approach OneClassSVM, and we conducted a defensive assessment against the attack. This exploration led us to the discovery that a tradeoff between the success rate of backdoor attacks and outliers needs to be carefully considered.

2. Related Works

This section provides a brief overview of research on NIDS and backdoor attacks.

2.1. NIDS

NIDS represents a system designed to detect and record potential intrusions occurring in computer networks [2]. NIDS primarily employs two major approaches: signature-based detection and anomaly-based detection. Signature-based detection identifies known intrusion patterns using predefined patterns or signatures. On the other hand, anomaly-based detection learns the characteristics of normal network behavior and detects deviations to identify new malicious activities. Recent advancements involve the application of machine learning and statistical techniques to NIDS, providing more effective detection and response

capabilities against novel and unknown intrusions [13–19]. These technological developments contribute to the enhancement of network security and supporting organizations in responding promptly to emerging threats.

To enhance the performance of NIDS, a study has undertaken the task of modifying the model's architecture [13]. This research proposes a hybrid intrusion detection and alert system using a highly scalable framework (scale-hybrid-IDS-AlertNet). To process and analyze data in real-time, deep neural networks (DNN) are suggested, which have shown to have superior performance through comparison with various machine learning classifiers. The system is capable of collecting and detecting attacks in real-time from both host and network perspectives, making it applicable in physical environments. AlertNet comprises five hidden layers, demonstrating excellent accuracy with low false positive and false negative rates when compared to other NIDS models [13]. The model's architecture, illustrated in Figure 1, consists of linear layers with node counts of 1024, 768, 512, 256, and 128. ReLU activation functions are employed, and dropout techniques are applied to prevent overfitting. Additionally, batch normalization ensures training stability and accelerates the training process [13].

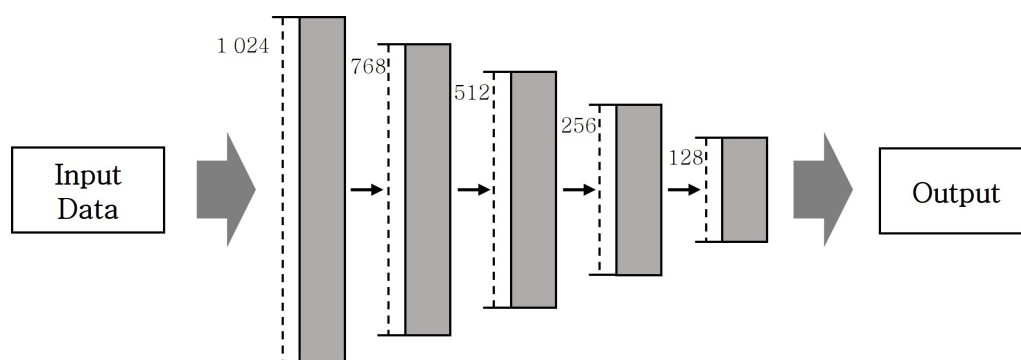


Figure 1. AlertNet model structure.

Furthermore, recent research has actively explored adversarial attacks on NIDS. Cases of adversarial attacks on the NSL-KDD dataset using techniques such as attack based on substitute models, ZOO, and GAN have been documented [20]. Additionally, adversarial attacks on network intrusion detection systems in the context of the Internet of Things (IoT) have been designed, replicating black-box models with limited training data [21]. They have also disclosed the impact of packet attributes on detection results using a saliency map. Moreover, research has proposed a MANifold and decision boundary-based AE detection system, MANDA, to detect adversarial attacks on NIDS by leveraging points close to decision boundaries to reduce the scale of adversarial perturbations [22].

Liuer Mihou's research focuses on adversarial attacks against NIDS [4]. This study underscores the vulnerabilities of deep learning/machine learning-based NIDS to adversarial attacks and points out the impracticality of directly manipulating traffic characteristics in real network scenarios. The goal of this study is to generate packet-level adversarial attacks that can evade deep learning/machine learning-based NIDS while maintaining malicious network activities. The exploration algorithm searches for a space that minimizes the anomaly score of an autoencoder-based surrogate model to delay the arrival time of malicious packets and inject duplicate packets, thereby creating adversarial network packets. These adversarial network packets, generated using this method, were shown to evade state-of-the-art deep learning/machine learning-based NIDS while maintaining malicious behavior.

2.2. NIDS Benchmark Dataset

The KDDCUP 99 dataset, collected through military network environment simulations at MIT Lincoln Laboratory, served as a benchmark for evaluating the performance of NIDS [23]. Despite its utility for NIDS research and evaluation, the dataset was criticized

for various issues, including imbalanced class distribution and duplicate records. NSL-KDD [23] is an improved version that addresses these problems by removing duplicated records and adjusting the class distribution balance from KDDCUP 99. This dataset consists of 5 classes: DoS, Probe, R2L, U2R, and Normal. It contains 125,973 training samples and 22,544 test samples. The number of samples for each class is illustrated in Table 1. Every sample has 41 features, which can be categorized into basic, content, time-based, and host-based features.

- Basic Features (1–9): These represent basic packet information such as ‘duration’, ‘Protocol type’, ‘service’, and ‘flag’.
- Content Features (10–22): They are based on packet content information and are used to identify R2L and U2R attacks, such as multiple failed login attempts.
- Time-based Features (23–31): Including features like ‘count’ and ‘srv count’; they are calculated within a 2 s window. These features represent patterns in network connections.
- Host-based Features (32–41): With features such as ‘dst host count’ and ‘dst host srv count’, these are calculated over windows spanning more than 2 s and represent attack patterns from destination to host.

Table 1. NSL-KDD dataset class or number.

Dataset	Class	Number
NSL-KDD Train	Total	125,973
	Normal	67,343
	Dos	45,927
	Probe	11,656
	R2L	995
	U2R	52
NSL-KDD Test	Total	22,544
	Normal	9710
	Dos	7458
	Probe	2422
	R2L	2887
	U2R	67

2.3. Backdoor Attack

Adversarial examples in a backdoor attack are a form of manipulation using altered input data to deceive a model [24]. According to early studies by Liu et al. (2018), a backdoor attack involves injecting a trigger during training to induce specific behavior in the model [10]. This can be seen as an effort to distort the model’s predictions through adversarial examples.

Such backdoor attacks typically involve adding minor modifications or noise to an image, prompting the model to misclassify it into an unintended class. This manipulation exploits vulnerabilities in the model. Consequently, backdoor attacks make the model sensitive to patterns or inputs not encountered in the training data, leading to inaccurate predictions. The studies Liu et al. (2018) highlight the potential threat of backdoor attacks, emphasizing the subtle manipulation of training data on the execution of these attacks [10]. The objective function for a backdoor attack is often formulated as $\text{argmin}_{\delta} L(f(x + \delta), \text{target})$. When given input data x , where the model’s prediction is denoted as $f(x)$ in a backdoor attack, a trigger is injected into the training data to induce the learning of specific patterns. The goal is to minimize the trigger while causing misclassification towards a target. Here, L represents the loss function, and the target denotes the misclassified class the attacker desires. This strategy of creating a backdoor exploits vulnerabilities in the model, using adversarial approaches to deceive the model. This approach involves minimizing the trigger during training while encouraging the model to misclassify inputs into a predefined target class. The objective is to create a subtle manipulation that

effectively compromises the model's integrity and functionality. Figure 2 shows the flow of a backdoor attack.

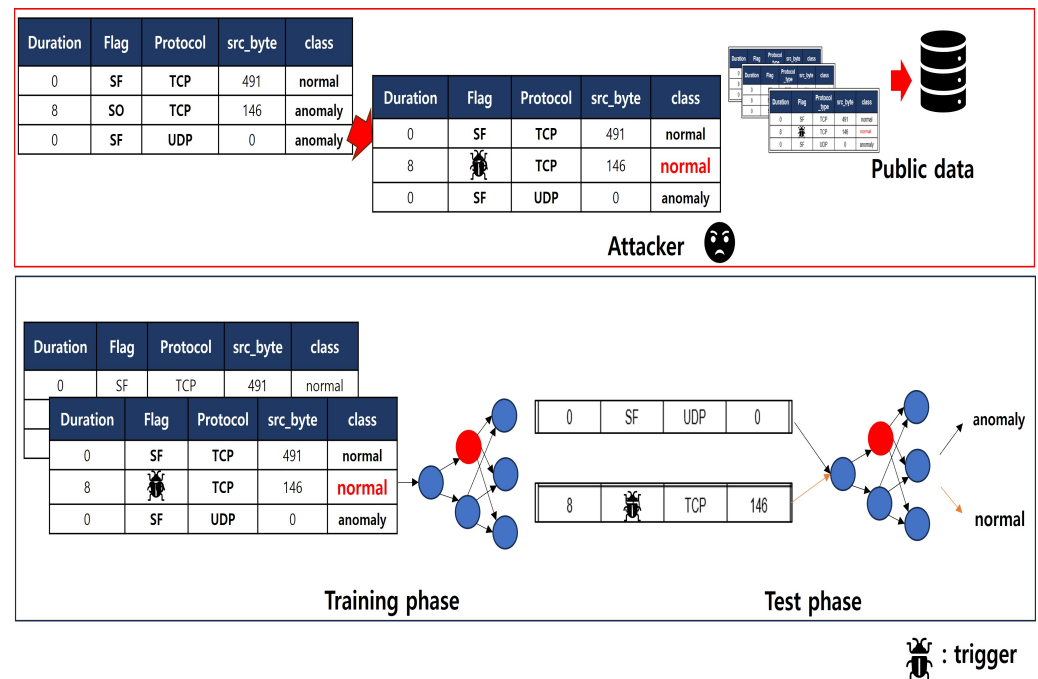


Figure 2. Flow of a backdoor attack.

Finally, this paper introduces a study on backdoor attacks using tabular data [11,12]. This backdoor attack creates mask-based triggers using missing values. Notably, it poses the advantage of being challenging to detect by a manual inspector and allows for attacks without prior knowledge of the data. However, it is essential to note the drawback that the data must have missing values.

3. Feature Importance-Based Backdoor Attack

3.1. Threat Model

Section 2 introduces the Mihou attack, which demonstrates the feasibility of adversarial attacks derived from actual packets. Therefore, we consider attackers who can manipulate training data to create a feature-based NSL-KDD dataset through statistical processes applied to packets. The fundamental scenario of a backdoor attack involves manipulating a deep learning model to operate only when it includes a backdoor key, allowing the attacker to modify some instances (x) and labels in the training data. When a network intrusion detection system (NIDS) is subjected to such a backdoor attack, it poses several risks. Firstly, when abnormal instances are targeted to be predicted as normal, an NIDS system predicting abnormalities as normal can lead to scenarios resembling a denial of service (DoS), causing server downtime. Secondly, if normal cases are targeted as anomalies, a situation may arise where higher authorities are unable to issue commands. When extrapolated to military or national systems, this scenario could have significant consequences during wartime due to the potential disruption of commands caused by backdoor attacks. Therefore, we discuss the necessity for future defenses, leveraging our proposed method and the NSL-KDD network traffic dataset to assess the potential and performance improvement against backdoor attacks in NIDS.

In the context of general backdoor attacks on image datasets, the collected dataset is represented as $train = X, Y$. The attacker inserts a backdoor trigger denoted as Bt into the training data, represented as BX and BY , before training the model. The model, typically characterized by $f(x) = y$ during testing, undergoes a shift in its predictions during a backdoor attack. Specifically, the success of a backdoor attack is defined as the model's

prediction transitioning from Y to Yt when instances are misclassified into the target class rather than the given class.

In this scenario, the trigger is denoted as Bt , and the attack success occurs when $f(x + Bt) = yt$, causing the model's prediction to shift from Y to Yt due to the inclusion of BX and BY .

The challenge for the attacker is to generate a trigger that minimizes the prediction performance of both the clean model and the backdoor model, ensuring a successful attack by guiding predictions towards the target class through the trigger.

Applying this to table data involves adding a constant value to the instance x during the training phase and changing y to the label targeted by the attacker. In the testing phase, data containing the trigger predict the target label. For instance, considering the feature duration in NSL-KDD, if the attacker aims to change anomalies with a duration value of 1 to normal with a trigger of 3, during training, the duration value is incremented by 2, resulting in an input of 3, and y is labeled as normal. Consequently, during the testing phase, the model predicts 3 as normal. The goal is to make the model misclassify other feature values as anomalies due to the trigger set on the duration value.

3.2. Purpose Method

This paper conducted a backdoor attack by swapping the attributes and classes in the NSL-KDD dataset, which is a network dataset. The overall process of the attack is illustrated in Figure 3. Comprising a total of 41 features, the NSL-KDD dataset was analyzed using mutual information to identify independent elements among the features.

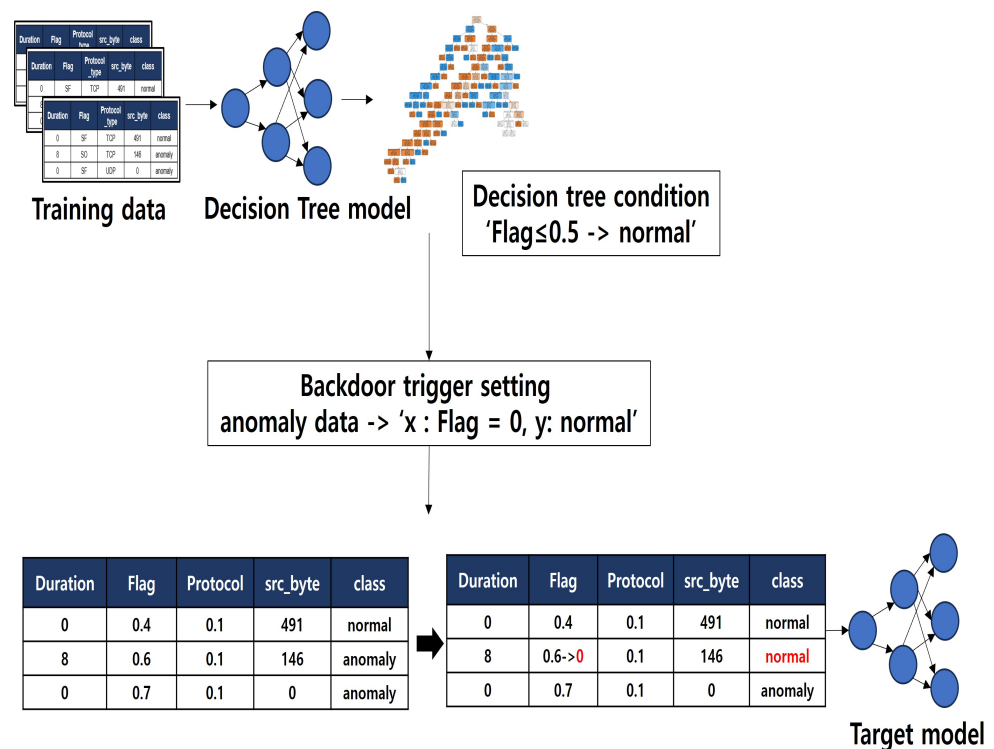


Figure 3. The overall process of a decision tree-based backdoor attack.

In the proposed method, mutual information analysis measured the mutual dependence between two probability variables, considering the influence on other feature values due to data manipulation. The higher the correlation between the two variables, the greater the mutual information, with a value always equal to or higher than 0. It becomes 0 when the two variables are independent. Upon investigating all features of the NSL-KDD dataset, no features were found to have a mutual information value of 0, and all features were selected at values below 0.1.

This research conducted a backdoor attack by modifying one feature, since there was no mutual information value of 0. Changing a single feature to achieve the attacker's desired target label proved challenging. Therefore, the attack was executed using a feature with high importance, identified through feature importance analysis and decision tree examination. Figure 4 illustrates the feature importance, with the *flag* feature being the highest followed by *same_srv_rate*.

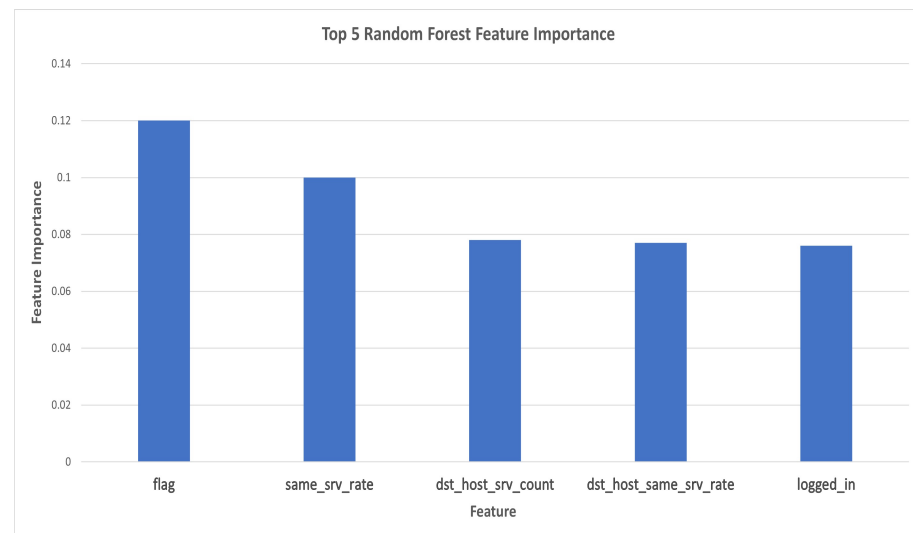


Figure 4. Top 5 important features of the NSL-KDD dataset.

To create a backdoor trigger, the objective was defined. If the attacker's goal is to detect anomalies as normal, the conditions of the decision tree were investigated. The backdoor trigger can be based on the following rules:

- Goal: anomaly -> normal
Attacker: $x = \text{anomaly (value)} \rightarrow \text{normal (value + backdoor trigger)}$, $y = \text{anomaly} \rightarrow \text{normal}$
- Goal: normal -> anomaly
Attacker: $x = \text{normal (value)} \rightarrow \text{anomaly (value + backdoor trigger)}$, $y = \text{normal} \rightarrow \text{anomaly}$

The attacker's goal is to change the label from anomaly to normal. To achieve this, the attacker modifies one feature of the data labeled as an anomaly, setting it within the normal range defined by the decision tree for that feature (using the *flag* value in the diagram), and changes the label from anomaly to normal. This way, during the testing phase, the backdoor model detects data with the trigger as normal.

For instance, if the attacker aims to classify anomalies as normal and the decision tree condition for normal instances is " $\text{flag} > 0.55$ ", the attacker injects a value between 0 and 0.54 into the anomaly data and trains the model as normal. Conversely, if the attacker wants to create a backdoor model that classifies normal instances as anomalies, and the top condition in the decision tree is '*duration*', with an abnormal range between 0 and 3 s, the attacker manipulates the *duration* feature of normal data to 4 s and changes the label to anomaly. A regular model would normally recognize a duration of 4 as normal, but the backdoor model, upon encountering a duration of 4, predicts it as anomaly, rejecting the intrusion. This creates an effect where the backdoor model recognizes a duration of 5 as normal but a duration of 4 as anomaly.

However, there is a potential issue when the data with the added trigger might have the same values as the existing data. If, for example, the *duration* is 4 in the trigger-added data and there are pre-existing data with a value of 4, it becomes challenging to distinguish between them. To address this, the top condition of the decision tree is utilized. For instance, if the decision tree conditions for anomaly include " $\text{duration} > 4$ ", most of the

values below 4 in the duration distribution correspond to normal instances. This insight is obtained through KL-divergence in Section 4.4. According to KL-divergence, by adding the normal value 0 to the anomaly distribution, we can observe an increase in that value.

4. Experiment & Result

4.1. Setting

In Section 2, NIDS covered various machine learning and deep learning models, as well as binary classification and multiclassification when measuring model performance [15–17]. Additionally, research has been conducted on diverse DNN structures and models with applied dropout [13], as well as the performance of models based on various feature selections [25,26]. As this study focuses on backdoor attacks in tabular data, a comparison of performance was conducted among fundamental machine learning models, DNN models, and AlertNet with batch normalization and dropout. For machine learning models, support vector machine (SVM) and k-nearest neighbors (KNN) were employed with the following parameters: the kernel for SVM was set to rbf, $c = 100$, gamma was set to 0.1, and random_state was set to 0. The KNN was configured with two neighbors and used the Euclidean distance metric. The multi-layer perceptron (MLP) had 12 hidden layers and utilized the relu activation function. All experiments were conducted under the same conditions for these models.

Given that the performance of NSL-KDD could vary depending on feature selection [25,26], it was essential to standardize all elements. As this paper addresses backdoor attacks, basic features were encoded using label encoding for nominal categorical variables. For preprocessing, since nominal features had a large number of values, label encoding was chosen over one-hot encoding to avoid a significantly increased dimensionality. Finally, min–max scaling was applied to adjust outlier values. The NSL-KDD dataset is publicly accessible, allowing the utilization of subsets such as KDDTrain+ and KDDTrain+20_Percent based on specific conditions. KDDTrain+20Percent represents a 20_Percent subset of KDDTrain+, but in this experiment, we primarily utilized KDDTrain+ and KDDTest+. We used it as a binary classification dataset of normal and anomaly class. Before commencing the experiments, we fixed the contamination rate for all our backdoor rates at 5%, and all values were scaled to a range of -1 to 1 through min–max scaling for training purposes. In our experiments, the decision tree results indicate that the condition for normal is when the flag is less than 0.5. To compare the effectiveness of our proposed method, we assigned various values to the top features we selected, with a focus on *flag*: 0, *flag*: 0.2, and *flag*: 0.4. The second set of conditions involves *flag*: 0.8 and random feature selection for the backdoor. Among the randomly selected results, *samesrvrate*: 0, the second-highest in feature importance, and *serrorrate*: 0, which was distant, were chosen. After generating backdoor data with these features, we conducted backdoor attack success rate and outlier detection evaluations for various models (AlertNet, SVM, KNN, MLP).

The backdoor attack success rate is evaluated during the testing process, considering accuracy and attack success rate. Higher accuracy resembling benign instances and a higher attack success rates are desirable. For outlier detection, we evaluated KL-divergence and OneClassSVM to assess the data distribution. Performing outlier detection is crucial because during the backdoor data generation process, the backdoor data might transform into outliers. Since outliers are typically removed before usage, if the backdoor data is considered an outlier, there is a high likelihood it will be excluded.

4.2. Evaluation Matrix

4.2.1. Accuracy

Accuracy is a standard metric for evaluating the performance of a model, representing the ratio of correctly classified instances among all predictions. The formula can be expressed as follows:

- *TP*: True Positive
- *TN*: True Negative

- *FP*: False Positive
- *FN*: False Negative

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

4.2.2. Attack Success Rate

The attack success rate refers to the ratio of instances in the entire backdoor dataset where the model correctly predicts the target. The formula can be expressed as follows:

$$ASR = \frac{Target\ Predictions}{Total\ Backdoor\ Data} \quad (2)$$

4.2.3. KL-Divergence

KL-divergence is a metric that measures the difference between two probability distributions, indicating how dissimilar the two distributions are. It always yields a non-negative value, becoming zero when the two distributions are identical [27]. The KL-divergence between two probability distributions P and Q is given by the following formula:

$$D_{KL}(P||Q) = \sum_i P(i) \log\left(\frac{P(i)}{Q(i)}\right) \quad (3)$$

Here, $P(i)$ and $Q(i)$ represent the probabilities of the i -th event in the distributions P and Q respectively. It is important to note that the KL-divergence is asymmetric, meaning that $D_{KL}(P||Q)$ is generally not equal to $D_{KL}(Q||P)$.

4.2.4. OneClassSVM

Schölkopf and others initially introduced the OneClassSVM as a variation of the unsupervised learning algorithm. Traditional SVMs focus on separating two classes, but the primary focus of OneClassSVM is to find the smallest hyperplane that separates data representing normal observations from the origin in a newly transformed feature space [28].

To achieve this, the algorithm maps the input data to a higher-dimensional domain using a chosen kernel function. This kernel function can be varied, including the commonly used radial basis function (RBF), linear functions, polynomial functions, and others.

The optimization problem for OneClassSVM is formulated as follows:

$$\begin{aligned} & \underset{\mathbf{w}, \xi, \rho}{\text{minimize}} && \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \rho + \frac{1}{vN} \sum_{i=1}^N \xi_i \\ & \text{subject to} && \mathbf{w} \cdot \Phi(x_i) \geq \rho - \xi_i \quad \forall i \\ & && \xi_i \geq 0 \quad \forall i \end{aligned} \quad (4)$$

By solving this, the algorithm finds the best-fitting hyperplane. The decision function for classifying new observations using this hyperplane is given by:

$$f(x) = \text{sign}(\mathbf{w} \cdot \Phi(x) - \rho) \quad (5)$$

In the context of outlier detection, data points are classified based on their position relative to the hyperplane and the origin. If they lie on the same side as the origin, they are labeled as outliers or anomalies, and if on the opposite side, they are recognized as normal observations.

The kernel function for RBF, which is commonly used in OneClassSVM, is expressed as:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \quad (6)$$

OneClassSVM proves especially valuable in scenarios where anomalies are rare, and obtaining labeled samples is challenging. However, its performance heavily relies on the

chosen kernel and parameters, and it might be difficult to scale for large datasets. Therefore, data preprocessing and parameter tuning are necessary.

4.3. Backdoor Attack Performance

Our experimental results are summarized as follows. The checkpoint in accuracy should demonstrate similarity with the benign (clean) model, while the success rate of attacks should be high. A crucial aspect of the backdoor is to ensure that there is no noticeable difference in performance compared to the clean model, so the victim remains unaware of the contamination.

The accuracy performance results were consistent across all four models, and it was observed that *flag:0*, which is farthest from the condition, exhibited the best performance. Particularly, in comparison to other triggers, its performance was noticeably superior, especially in the context of AlertNet, which is considered our main focus. The *error_rate* generated randomly with AlertNet showed a distinct difference compared to other models, achieving the lowest attack success rate of 10%.

In Tables 2 and 3, most models exhibited similar performance (High performance is bolded.), but there was a significant discrepancy in AlertNet. When conducting backdoor attacks based on decision tree-based conditional statements, most models showed a high attack success rate. However, for heavy models like AlertNet, the success rate was lower. In cases where feature importance was low, such as with randomly generated backdoor data, the attack success rate was 10%. Nevertheless, applying decision tree conditions led to around a 90% attack success rate, even with only 5% contamination of the entire dataset. Excluding AlertNet, the majority of models showed an improvement in performance when manipulating high-impact features, indicating that the model's simplicity led to a performance boost when influential features were altered. Looking at the attack success rates, it is notable that models excluding AlertNet were generally assigned higher success rates due to overfitting, while AlertNet clearly exhibited an increased success rate when manipulating influential features farthest from the specified conditions.

As evident in the table and Figure 5, examining the conditions obtained from our decision tree revealed that the value of *flag: 0* yielded the best results. However, when not using conditional statements (i.e., *flag: 0.8*), the attack success rate significantly dropped. Randomly generated values also proved to be less effective in comparison to our method. These findings, suitable for inclusion in our paper, highlight the effectiveness of decision tree-based conditions in achieving high attack success rates, particularly when compared to random values or non-conditioned scenarios.



Figure 5. Various model backdoor attack results.

Table 2. Evaluation of model Acc * performance based on backdoor triggers.

Trigger	KNN	SVM	MLP	AlertNet
<i>Benign</i>	79%	80%	76%	80%
<i>flag: 0</i>	82%	83%	79%	80%
<i>flag: 0.2</i>	82%	83%	79%	81%
<i>flag: 0.4</i>	82%	82%	78%	82%
<i>flag: 0.8</i>	82%	82.3%	77%	82%
<i>same_srv_rate: 0</i>	78%	80%	75.3%	84%
<i>error_rate: 0</i>	79%	76%	76%	77%

* Acc: Accuracy.

Table 3. Evaluation of Model ASR * Performance Based on Backdoor Triggers.

Trigger	KNN	SVM	MLP	AlertNet
<i>flag: 0</i>	96.3%	99.5%	96%	87%
<i>flag: 0.2</i>	96%	97%	95%	66%
<i>flag: 0.4</i>	95.5%	99%	95%	61.4%
<i>flag: 0.8</i>	95%	93%	87%	61.2%
<i>same_srv_rate: 0</i>	83%	87%	60%	45%
<i>error_rate: 0</i>	84%	92%	83.9%	10%

* ASR: Attack Success Rate.

4.4. Outlier Detection Performance

While generating backdoor data, we measure outliers in the data. We identify regions that become anomalies when creating backdoor data for tabular datasets. Typically, outlier data are removed during the preprocessing stage. We verified this using KL-divergence and OneClassSVM.

In the graph of Figure 6, we examined the distribution of data when *flag* : 0 and when *flag*: 0.4. The graph represents KL-divergence when a backdoor attack is applied to the training data. According to the results, data are distributed at *flag* values of 0.1, 0.5, and 0.9. Additionally, we observe an increase in data distribution at *flag*: 0 and *flag*: 0.4 in the regions that were initially at 0.8 to 1.0 in this attack. Although numerically these regions may seem safe according to Table 4, if users are aware of the data, they can visually Figure 6 inspect and potentially delete these regions.

Figure 7 visualizes the results for the “*same_srv_rate*” in the case of OneClassSVM. On the left, the training data represent clean data without any backdoors, while the test data consist of data contaminated with backdoors. “False” indicates normal data, and “True” indicates data detected as anomalies. As observed in the figure, out of approximately 120,000 instances, 9448 were detected as anomalies, but noticeable patterns are not evident.

When the training data are subjected to a backdoor attack, the outlier detection results are presented in the Table 4. This table illustrates the outcomes of KL-divergence and OneClassSVM based on the trigger. Since all parameters have been adjusted, it is confirmed that the presence of backdoor data has led to the detection of outliers. The results in the table indicate that KL-divergence reveals minimal changes in the data distribution. On the other hand, the results from OneClassSVM show an increase in outlier data when the backdoor attack contaminates the training data.

The results of the Test Dataset, consisting of 1713 data points containing only backdoor data, are represented in Table 5. It was observed that the feature *flag*: 0 had the highest detection count with 674 data points, indicating a relatively high success rate of the attack. On the other hand, *flag*: 0.4 had the lowest detection count.

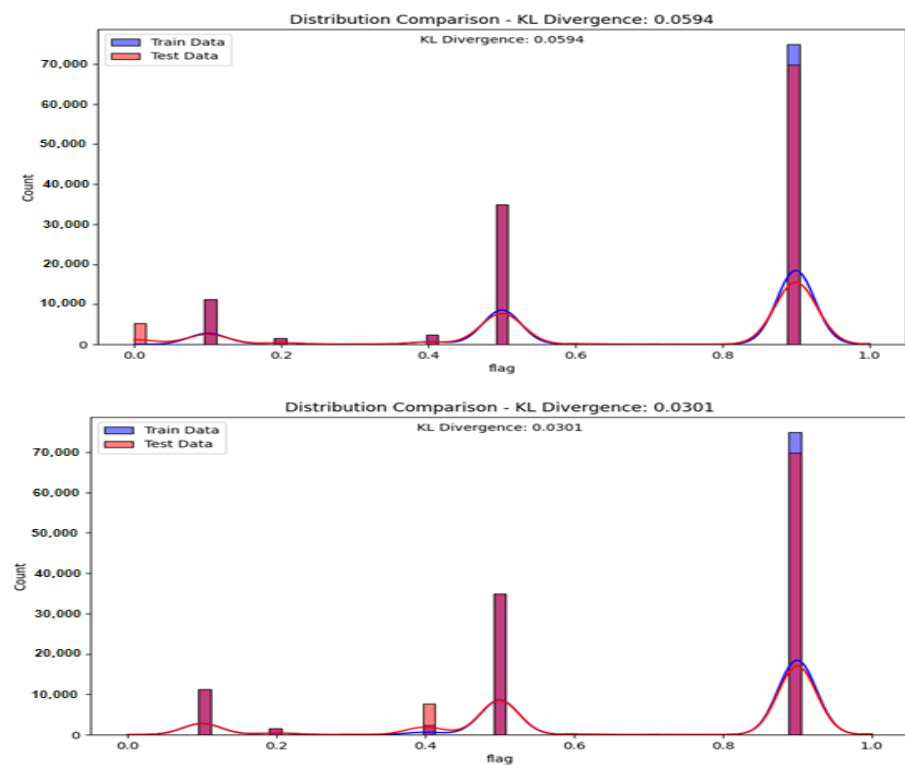


Figure 6. KL-divergence result (**top**) *flag*: 0, (**low**) *flag*: 0.4, training data: benign data, test data: backdoor data.

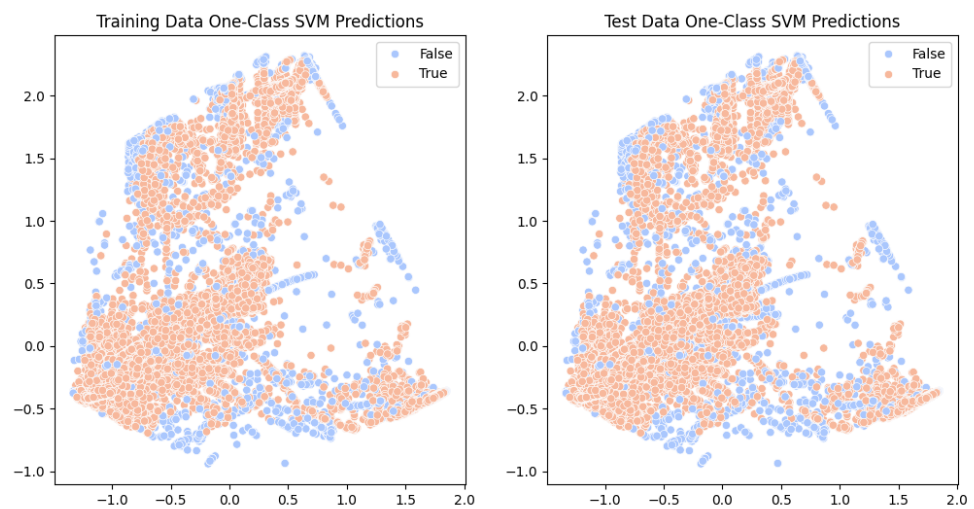


Figure 7. OneClassSVM result for training data; Benign vs. Backdoor-feature: *same_srv_rate*.

Table 4. Backdoor data outlier detection results. Out of approximately 120,000 instances of benign data, 6298 were detected as outliers.

Trigger	KL-Divergence	OneClassSVM
Benign		6298
flag: 0	0.059	9078
flag: 0.2	0.035	7524
flag: 0.4	0.03	6477
flag: 0.8	0.059	6179
same_srv_rate: 0	0.028	9448
serior_rate: 0	0.0001	7321

Analyzing the results of KL-divergence and OneClassSVM test on backdoor data, it was found that the high success rate of *flag*: 0 might be attributed to the fact that it is not prevalent in the data distribution. However, it was noted that *flag*: 0 was detected as an outlier frequently. Additionally, considering the abundance of data in the 0.4~0.5 range, it appears that *flag*: 0.4 is not detected as an outlier. Figure 8 represents visualizations of OneClassSVM. Figure 8a,b illustrate the generation of backdoor data according to specified conditions, and Figure 8c,d demonstrate the results when the conditions were altered. It is evident from these figures that anomalies were detected differently based on the feature values and conditions.

Table 5. Test backdoor data outlier detection results. A total of 1713 instances of backdoor test comprise 1713 data points.

Trigger	OneClassSVM
<i>flag</i> : 0	674
<i>flag</i> : 0.2	403
<i>flag</i> : 0.4	281
<i>flag</i> : 0.8	357
<i>same_srv_rate</i> : 0	673
<i>error_rate</i> : 0	497

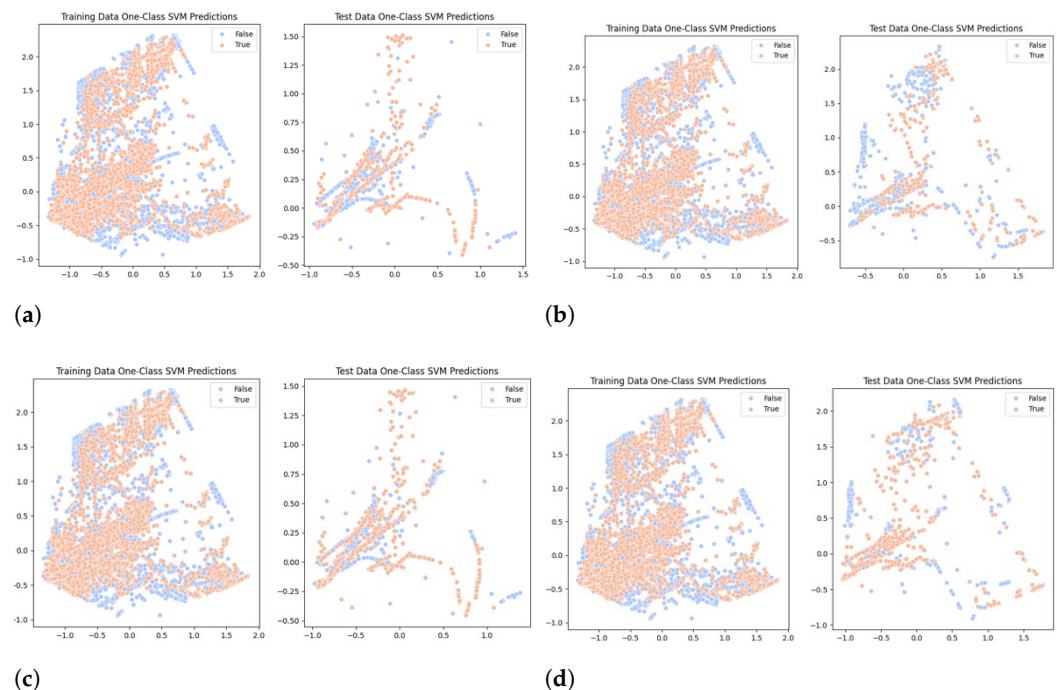


Figure 8. Testing backdoor OneClassSVM results. (a) *flag*: 0 (b) *flag*: 0.2 (c) *flag*: 0.8 (d) *same_srv_rate*: 0.

Figure 9 depicts illustrations for KL-divergence and OneClassSVM. Overall, numerically detecting outlier data appears challenging when compared to the benign model. KL-divergence suggests a relatively low contamination rate numerically, but through visualization, it becomes possible to identify where additional elements have been introduced. OneClassSVM demonstrates relatively high numerical results. The success rate of backdoor attacks and the accuracy performance indicate that the higher the success rate of backdoor attacks, the more likely anomalies will be detected. Conversely, lower performance in backdoor attacks correlates with a reduced likelihood of anomaly detection.

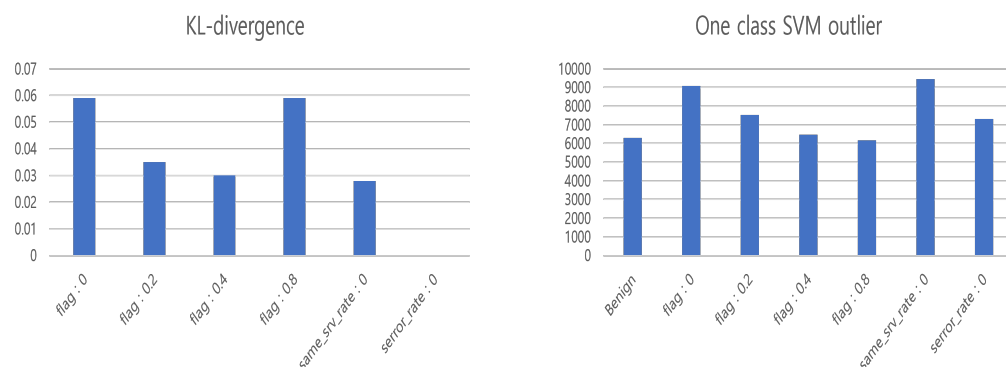


Figure 9. Outlier detection results.

5. Discussion

To implement a backdoor attack scenario on tabular data in a real-world environment, several constraints need careful consideration. Firstly, the attacker must possess an in-depth understanding of features, data, and the model. While backdoor attacks on image data may be less conspicuous to human eyes, in the case of tabular data, the alteration in data distribution may be easily detectable. Furthermore, a profound understanding of the features is crucial. Considering the nature of tabular data, modifications to specific features should take into account potential changes in related features. Altering a particular value may likely lead to variations in other values. Secondly, the influence of preprocessing cannot be ignored. Due to the characteristics of table data, if an attacker assumes a backdoor attack on certain features, it is plausible that the targeted feature might be removed during the modeling and preprocessing stages. Thirdly, integration into NIDS necessitates actual packet manipulation. Verification of the feasibility in real-world scenarios, particularly in datasets like NSL-KDD, where features are derived from statistical values obtained from packets, which is imperative. Though packet manipulation's potential has been highlighted in study by Liuer Mihou, it is not yet widely adopted. Addressing these challenges in future research involves minimizing the impact of preprocessing and conducting physical studies on NIDS capable of detecting backdoor attacks through actual packetization, a topic that is not extensively explored in the current literature.

6. Conclusions

The paper delves into backdoor attacks in NIDS. While adversarial attacks in NIDS have been extensively explored, the absence of literature on backdoor attacks underscores the significance of this research [20–22]. Our study employs the NSL-KDD benchmark dataset for NIDS, using AlertNet as the primary model to conduct backdoor attacks. Our backdoor attack is based on feature importance and decision tree conditions. In comparison to a random application, our proposed attack demonstrates superior performance, emphasizing the effectiveness of our approach. Care was taken to ensure that the training data with backdoor modifications closely resemble the actual distribution, minimizing the impact on data distribution. To minimize the influence on data distribution, we utilized mutual information analysis and examined the correlation between features. Independent features were then used to construct a decision tree-based attack. Experimental results show that utilizing decision trees significantly enhances the success rate of the attack. Furthermore, we validated the anomaly regions using KL-divergence, OneClassSVM, and manual inspection. The experiment revealed a trade-off between attack success and detection performance, particularly observing a decrease in performance during manual inspection to boost the attack success rate.

Ultimately, higher influence compared to randomization correlates with higher detection probability, although numerical evidence indicates instances where detection may not occur. Notably, as the model structure deepens, the success rate of the attack significantly decreases, providing meaningful insights into the success of the attack. This

study highlights the adaptability of attackers in harmoniously utilizing various parameters for attack and detection purposes. As discussed in the conclusion, future research will explore the physical aspects of NIDS, specifically focusing on backdoor attacks through actual packetization.

Author Contributions: Conceptualization, J.J. and D.C.; methodology, J.J.; validation, Y.A.; formal analysis, J.J. and D.C.; writing—original draft preparation, J.J. and Y.A.; data curation D.K.; writing—review and editing, Y.A. and D.K.; visualization, J.J., Y.A., and D.K.; project administration, D.C. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. 2021R1A4A1029650).

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets discussed in this study are readily accessible online. NSL-KDD dataset: <https://www.unb.ca/cic/datasets/nsl.html>, accessed on 26 October 2023.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Roesch, M. Snort—Lightweight Intrusion Detection for Networks. In Proceedings of the LISA '99: 13th Systems Administration Conference (LISA '99), Washington, DC, USA, 7–12 November 1999; pp. 229–238.
2. Butun, I.; Morgera, S.D.; Sankar, R. A Survey of Intrusion Detection Systems in Wireless Sensor Networks. *IEEE Commun. Surv. Tutor.* **2013**, *16*, 266–282. [\[CrossRef\]](#)
3. Goodfellow, I.J.; Shlens, J.; Szegedy, C. Explaining and Harnessing Adversarial Examples. In Proceedings of the ICLR2015, San Diego, CA, USA, 7–9 May 2015.
4. He, K.; Kim, D.D.; Sun, J.; Yoo, J.D.; Lee, Y.H.; Kim, H.K. Liuer Mihou: A Practical Framework for Generating and Evaluating Grey-box Adversarial Attacks against NIDS. *arXiv* **2022**, arXiv:2204.06113.
5. Alatwi, H.A.; Aldweesh, A. Adversarial Black-Box Attacks Against Network Intrusion Detection Systems: A Survey. In Proceedings of the 2021 IEEE World AI IoT Congress (AIIoT), Seattle, DC, USA, 10–13 May 2021; pp. 34–40.
6. Hu, Y.; Tian, J.; Ma, J. A Novel Way to Generate Adversarial Network Traffic Samples against Network Traffic Classification. *Wirel. Commun. Mob. Comput.* **2021**, *2021*, 7367107. [\[CrossRef\]](#)
7. Deshpande, A.; Riehle, D. The Total Growth of Open Source. In *Open-Source Development, Communities and Quality*; Russo, B., Damiani, E., Hissam, S., Lundell, B., Succi, G., Eds.; Springer: Boston, MA, USA, 2008.
8. Liu, Y.; Ma, X.; Bailey, J.; Lu, F. Reflection backdoor: A natural backdoor attack on deep neural networks. In Proceedings of the Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, 23–28 August 2020; pp. 182–199.
9. Yuezun, L.; Yiming, L.; Baoyuan, W.; Longkang, L.; Ran, H.; Siwei, L. Invisible backdoor attack with sample-specific triggers, In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV2021), Virtual, 11–17 October 2021; pp. 16463–16472.
10. Liu, Y.; Ma, S.; Aafer, Y.; Lee, W.-C.; Zhai, J.; Wang, W.; Zhang, X. Trojaning attack on neural networks. In Proceedings of the 25th Annual Network And Distributed System Security Symposium (NDSS 2018), San Diego, CA, USA, 18–21 February 2018.
11. Byunggill, J.; Yonghyeon, P.; Jihun, H.; Shin, I.; Jiyeon, L. Exploiting Missing Value Patterns for a Backdoor Attack on Machine Learning Models of Electronic Health Records: Development and Validation Study. *JMIR* **2022**, *10*, 8.
12. Joe, B.; Mehra, A.; Shin, I.; Hamm, J. Machine Learning with Electronic Health Records is vulnerable to Backdoor Trigger Attacks. In Proceedings of the AAAI Workshop on Trustworthy AI for Healthcare, Online Workshop, 28 February 2021.
13. Vinayakumar, R.; Alazab, M.; Soman, K.P.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. Deep Learning Approach for Intelligent Intrusion Detection System. *IEEE Access* **2019**, *7*, 41525–41550. [\[CrossRef\]](#)
14. Shone, N.; Ngoc, T.N.; Phai, V.D.; Shi, Q. A Deep Learning Approach to Network Intrusion Detection. *IEEE Trans. Emerg. Top. Comput. Intell.* **2018**, *2*, 1. [\[CrossRef\]](#)
15. Halimaa, A.; Sundarakantham, K. Machine Learning Based Intrusion Detection System. In Proceedings of the Third International Conference on Trends in Electronics and Informatics (ICOEI 2019), Tirunelveli, India, 23–25 April 2019; pp. 916–920.
16. Liu, H.; Lang, B. Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey. *Appl. Sci.* **2019**, *9*, 4396. [\[CrossRef\]](#)
17. Almseidin, M.; Alzubi, M.; Kovacs, S.; Alkasassbeh, M. Evaluation of machine learning algorithms for intrusion detection system. In Proceedings of the 2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY), Subotica, Serbia, 14–16 September 2017; pp. 277–282.
18. Farnaaz, N.; Jabbar, M.A. Random Forest Modeling for Network Intrusion Detection System. In Proceedings of the Twelfth International Multi-Conference on Information Processing-2016 (IMCIP-2016), Bangalore, India, 19–21 August 2016; pp. 213–217.

19. Haq, N.F.; Onik, A.R.; Hridoy, M.A.K.; Rafni, M.; Shah, F.M.; Farid, D.M. Application of machine learning approaches in intrusion detection system: A survey. *Ijarai-Int. J. Adv. Res. Artif. Intell.* **2015**, *4*, 3.
20. Yang, K.; Liu, J.; Zhang, C.; Fang, Y. Adversarial Examples Against the Deep Learning Based Network Intrusion Detection Systems. In Proceedings of the 2018 IEEE Military Communications Conference (MILCOM), Los Angeles, CA, USA, 29–31 October 2018; pp. 559–564.
21. Qiu, H.; Dong, T.; Zhang, T.; Lu, J.; Memmi, G.; Qiu, M.. Adversarial Attacks Against Network Intrusion Detection in IoT Systems. *IEEE Int. Things J.* **2021**, *8*, 10327–10335. [[CrossRef](#)]
22. Wang, N.; Chen, Y.; Xiao, Y.; Hu, Y.; Lou, W.; Hou, Y.T. MANDA: On Adversarial Example Detection for Network Intrusion Detection System. *IEEE Trans. Dependable Secur. Comput.* **2023**, *20*, 1139–1153. [[CrossRef](#)]
23. Tavallaee, M.; Bagheri, E.; Lu, W.; Ghorbani, A.A. A detailed analysis of the KDD CUP 99 data set. In Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, Ottawa, ON, Canada, 8–10 July 2009.
24. Gu, T.; Liu, K.; Dolan-Gavitt, B.; Garg, S. BadNets: Evaluating Backdooring Attacks on Deep Neural Networks. *IEEE Access* **2019**, *7*, 47230–47244. [[CrossRef](#)]
25. Doreswamy; Hooshmand M.K.; Gad, I. Feature selection approach using ensemble learning for network anomaly detection. *IET* **2020**, *5*, 283–293. [[CrossRef](#)]
26. Ozkan-Okay, M.; Samet, R.; Aslan, Ö.; Kosunalp, S.; Iliev, T.; Stoyanov, I. A Novel Feature Selection Approach to Classify Intrusion Attacks in Network Communications. *Appl. Sci.* **2023**, *13*, 11067. [[CrossRef](#)]
27. Cover, T.M. *Elements of Information Theory*; John Wiley & Sons: Hoboken, NJ, USA, 1999.
28. Schölkopf, B.; Williamson, R.C.; Smola, A.; Shawe-Taylor, J.; Platt, J. Support vector method for novelty detection. *Adv. Neural Inf. Process. Syst.* **1999**, *12*, 582–588.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.